



**Vidarbha Youth Welfare Society's
Sant Gadge Baba Amravati University**
Amravati, Maharashtra

**Industrial Internship Report on
“Console-Based Banking Information System”**

Prepared by
Purva Adsod
Core Java Internship
Summer 2026

Internship Provided by
upSkill Campus (USC) & The IoT Academy
in collaboration with
UniConverge Technologies Pvt. Ltd. (UCT)

Executive Summary

This report provides details of the Industrial Internship provided by upSkill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks' time. My project was a Console-Based Banking Information System — a Core Java prototype that replicates the essential functionalities of a real banking application: user registration, account management, deposits and withdrawals, fund transfer, password-protected login, account statements, and basic error handling — delivered through a menu-driven console interface with file-based data persistence. This internship gave me a very good opportunity to get exposure to industrial problems and design/implement a solution. It was an overall great experience.

TABLE OF CONTENTS

1	Preface.....	3
2	Introduction.....	4
2.1	About UniConverge Technologies Pvt Ltd.....	4
2.2	About upSkill Campus.....	5
2.3	The IoT Academy.....	5
2.4	Objectives.....	5
2.5	Reference.....	6
2.6	Glossary.....	6
3	Problem Statement.....	7
4	Existing and Proposed Solution.....	8
5	Proposed Design / Model.....	9
5.1	High Level Diagram.....	9
5.2	Low Level Diagram.....	9
5.3	Interfaces / Screenshots.....	10
6	Performance Test.....	12
6.1	Test Plan / Test Cases.....	12
6.2	Test Procedure.....	13
6.3	Performance Outcome.....	13
7	My Learnings.....	14
8	Future Work Scope.....	15

1. Preface

This report summarizes the work carried out during my six-week Core Java internship under upSkill Campus (USC), The IoT Academy and UniConverge Technologies Pvt. Ltd. (UCT), during which I designed, developed, tested and documented a Console-Based Banking Information System as my project deliverable.

Internships bridge the gap between classroom learning and industry practice. They give students the opportunity to apply theoretical programming concepts to a real problem statement, follow a project timeline, and produce a deliverable the way industry code is reviewed — which is why internships are such an important part of career development.

My project, the Banking Information System, is a prototype that replicates the core functionality of a real banking application. It supports user registration with a unique account number, password-protected login, deposits, withdrawals, fund transfers between accounts, and a printable account statement showing complete transaction history — all built using Core Java with file-based persistence.

I am thankful to upSkill Campus, The IoT Academy and UniConverge Technologies Pvt. Ltd. for providing this internship opportunity and for giving students like me a structured, industry-style problem statement to work on.

How the Program was Planned

Working on this project strengthened my understanding of object-oriented programming, exception handling, collections and file I/O in Java, and gave me practical experience in converting a written problem statement into a fully working, menu-driven application. Overall, the internship was a valuable and enjoyable learning experience.

I sincerely thank the mentors and coordinators at upSkill Campus, The IoT Academy and UniConverge Technologies Pvt. Ltd. for their guidance, and my faculty and peers for their support throughout this internship.

To my juniors and peers taking up this internship: start early, build the project feature by feature against the problem statement's checklist, and take screenshots of every working feature as you go — it will save you a lot of time when writing the report.

2. Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in the Digital Transformation domain, providing industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various cutting edge technologies e.g. Internet of Things (IoT), Cyber Security, Cloud Computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front End etc.

UCT IoT Platform — UCT Insight

UCT Insight is an IoT platform designed for quick deployment of IoT applications while providing valuable insights for business processes. Built in Java for backend and ReactJS for frontend.

- Device connectivity via MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- Supports cloud and on-premises deployments
- Build your own dashboard, Analytics, Alerts, Third-party integrations, Rule Engine

Smart Factory Platform — Factory Watch

- Scalable solution for production and asset monitoring
- OEE and predictive maintenance scaling up to digital twin
- Modular SaaS architecture to save time, cost and money

LoRaWAN Based Solutions

UCT is one of the early adopters of LoRaWAN technology, providing solutions in Agritech, Smart Cities, Industrial Monitoring, Smart Street Lighting, and Smart metering.

Predictive Maintenance

UCT provides industrial machine health monitoring and predictive maintenance solutions leveraging Embedded Systems, Industrial IoT and Machine Learning technologies.

2.2 About upSkill Campus (USC)

upSkill Campus, along with The IoT Academy and in association with UniConverge Technologies, has facilitated the smooth execution of the complete internship process. USC is a career development platform that delivers personalized executive coaching in a more affordable, scalable and measurable way.

Website: <https://www.upskillcampus.com/>

2.3 The IoT Academy

The IoT Academy is the EdTech division of UCT that runs long executive certification programs in collaboration with EICT Academy, IIT Kanpur, IIT Roorkee and IIT Guwahati in multiple domains.

2.4 Objectives of this Internship Program

The objectives of this internship program were to:

- 🔗 Get practical experience of working in the industry.
- 🔗 Solve real-world problems.
- 🔗 Have improved job prospects.
- 🔗 Have an improved understanding of our field and its applications.
- 🔗 Achieve personal growth such as better communication and problem-solving skills.

2.5 Reference

- [1] Oracle Java Documentation – <https://docs.oracle.com/en/java/>
- [2] upSkill Campus – <https://www.upskillcampus.com/>
- [3] GeeksforGeeks, Java File Handling – <https://www.geeksforgeeks.org/file-handling-java/>
- [4] GitHub Repository – <https://github.com/purvaadsod/Banking-Application>

2.6 Glossary

Terms	Acronym
UCT	UniConverge Technologies Pvt. Ltd.
USC	upSkill Campus
JDK	Java Development Kit
OOP	Object-Oriented Programming
IDE	Integrated Development Environment
I/O	Input/Output
CLI	Command Line Interface

3. Problem Statement

In the assigned problem statement, I was required to develop a prototype of a Banking Information System in Core Java that provides a working preview of the key functionalities of a real banking system. The prototype had to demonstrate:

- User registration with a unique auto-generated account number
- Account management with balance tracking
- Deposit and withdrawal operations with validation
- Fund transfer between registered accounts
- Password-protected secure login system
- Viewable account statement with complete transaction history and timestamps
- Graceful error handling for insufficient funds, invalid inputs, and non-existent accounts
- Simple console interface with file-based data persistence between application runs

4. Existing and Proposed Solution

Existing solutions in this space range from full-fledged core banking software used by banks (e.g. Finacle, BaNCS) to everyday net-banking and mobile-banking apps. These are enterprise-grade systems built on layered architectures with databases, security modules and regulatory compliance — not accessible to students for learning.

My proposed solution is a lightweight, console-based Banking Information System built entirely in Core Java. It models the essential entities of a bank as Java classes, uses collections to manage accounts in memory, and persists data to local files using Java File I/O so data survives between application runs without needing an external database.

The value addition is a complete banking workflow — registration, login, deposit, withdrawal, transfer and statement generation — with full input validation and error handling, kept simple enough to be fully understood and extended by a single developer.

Code Submission (GitHub Link)

GitHub Repository: <https://github.com/purvaadsod/Banking-Application>

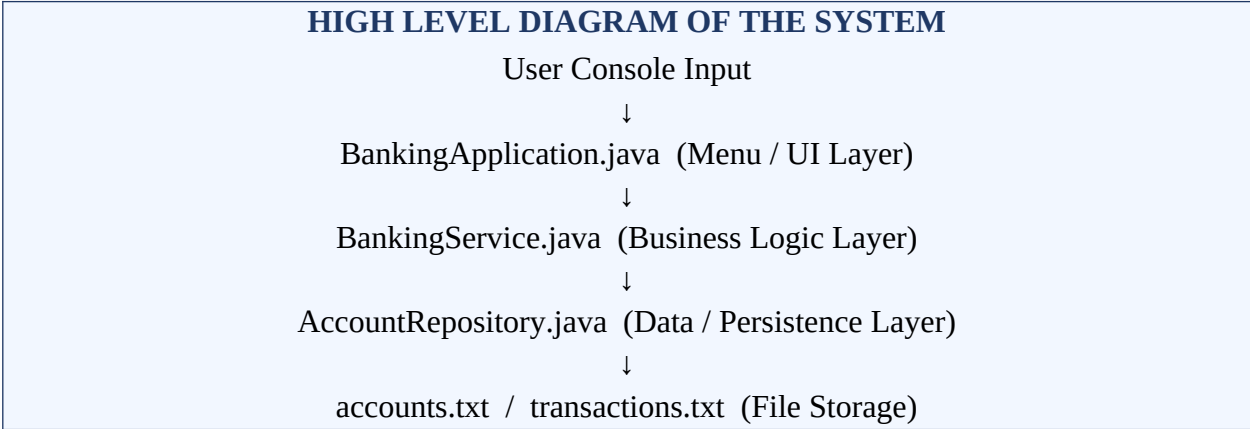
Report Submission (GitHub Link)

Report PDF uploaded to: <https://github.com/purvaadsod/Banking-Application>

5. Proposed Design / Model

The Banking Information System follows a simple layered design. A UserAccount model class holds all account details and balance; a BankingService class implements the business logic; a Transaction class records every operation; and a console-based BankingApplication class handles user interaction. Data flows from user input through validation and business-logic checks, to updates in the in-memory account list, which are then persisted to file.

5.1 High Level Diagram



5.2 Low Level Diagram

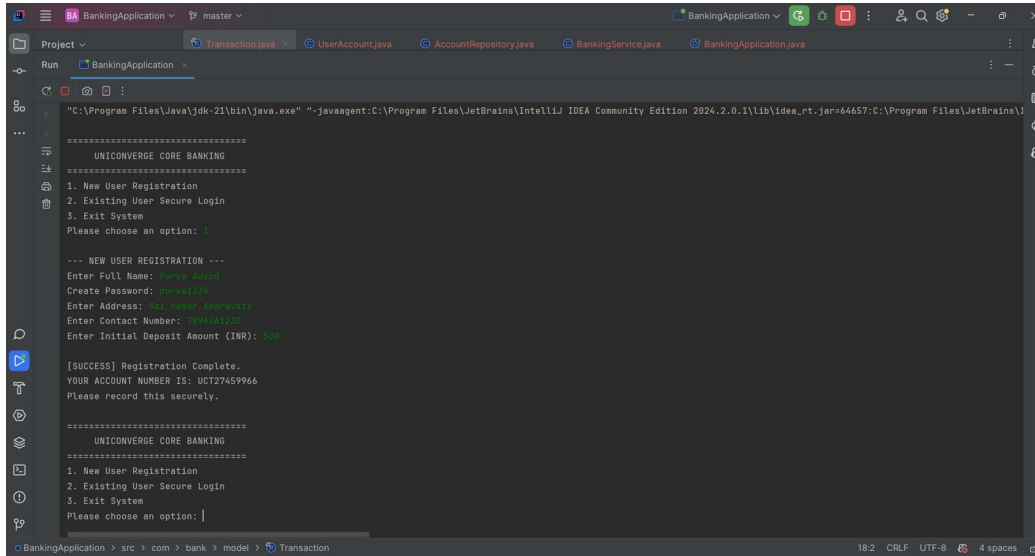
At a lower level, each menu option maps to a dedicated method in the service class:

Menu Option	Method Called	Output
New User Registration	registerUser()	Unique account number generated, data saved to file
Secure Login	login()	Session authenticated with account number + password
Deposit Funds	deposit()	Balance updated, transaction recorded
Withdraw Funds	withdraw()	Balance checked, updated, transaction recorded
Fund Transfer	transfer()	Sender debited, recipient credited, both updated
Print Statement	printStatement()	All transactions listed with timestamp and balance
View/Update Profile	viewProfile()	Account details displayed or updated

5.3 Interfaces / Screenshots

The only interface is the console (standard input/output). All interaction happens through numbered menu options and prompts. The following screenshots demonstrate the working application:

Screenshot 1: New User Registration

A screenshot of an IDE showing the execution of a Java application. The console output displays the 'UNICONVERGE CORE BANKING' menu with options 1 (New User Registration), 2 (Existing User Secure Login), and 3 (Exit System). Option 1 is selected, leading to a 'NEW USER REGISTRATION' section. The user enters: Full Name: Purva Adsod, Create Password: purva1234, Enter Address: Sai Nagar, Amravati, Enter Contact Number: 7894561230, and Enter Initial Deposit Amount (INR): 500. The system confirms '[SUCCESS] Registration Complete' and provides the account number 'UCT27459966'. The console then returns to the main menu.

```
=====
UNICONVERGE CORE BANKING
=====
1. New User Registration
2. Existing User Secure Login
3. Exit System
Please choose an option: 1

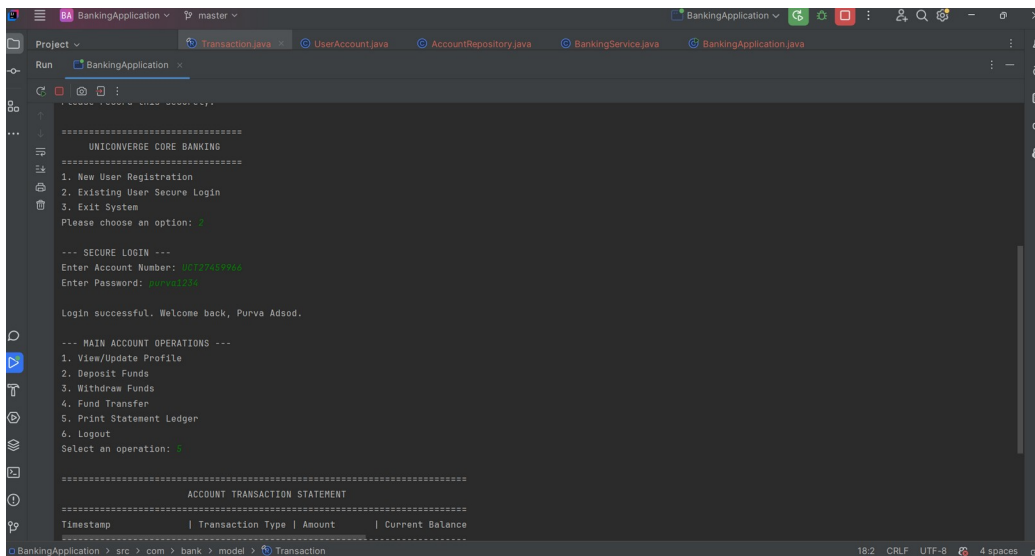
--- NEW USER REGISTRATION ---
Enter Full Name: Purva Adsod
Create Password: purva1234
Enter Address: Sai Nagar, Amravati
Enter Contact Number: 7894561230
Enter Initial Deposit Amount (INR): 500

[SUCCESS] Registration Complete.
YOUR ACCOUNT NUMBER IS: UCT27459966
Please record this securely.

=====
UNICONVERGE CORE BANKING
=====
1. New User Registration
2. Existing User Secure Login
3. Exit System
Please choose an option: |
```

Successful new user registration: Name — Purva Adsod, Address — Sai nagar, Amravati, Contact — 7894561230, Initial Deposit — Rs 500. System generated unique account number: UCT27459966.

Screenshot 2: Secure Login & Main Menu

A screenshot of the IDE console showing the application after a successful login. The user enters account number 'UCT27459966' and password 'purva1234'. The system responds with 'Login successful. Welcome back, Purva Adsod.' and presents the 'MAIN ACCOUNT OPERATIONS' menu with options 1 through 6. Option 3 is selected, leading to the 'ACCOUNT TRANSACTION STATEMENT' section, which displays a table of transactions.

```
--- SECURE LOGIN ---
Enter Account Number: UCT27459966
Enter Password: purva1234

Login successful. Welcome back, Purva Adsod.

--- MAIN ACCOUNT OPERATIONS ---
1. View/Update Profile
2. Deposit Funds
3. Withdraw Funds
4. Fund Transfer
5. Print Statement Ledger
6. Logout
Select an operation: 3

=====
ACCOUNT TRANSACTION STATEMENT
=====
Timestamp | Transaction Type | Amount | Current Balance
=====
```

Successful login using account number UCT27459966 and password, with the Main Account Operations menu showing all available features.

Screenshot 3: Account Transaction Statement

TC 1	Register new user	Valid name, address, contact, deposit	Account created with unique account number
TC 2	Login with correct credentials	Valid account number + password	Access granted, welcome message shown
TC 3	Login with wrong password	Valid account number, wrong password	Access denied, error message shown
TC 4	Deposit valid amount	Positive deposit amount	Balance increases, confirmation shown
TC 5	Withdraw more than balance	Amount > current balance	Transaction rejected, insufficient funds error
TC 6	Transfer to valid account	Valid recipient account number	Both balances update correctly
TC 7	Transfer to invalid account	Non-existent account number	Transaction rejected, no balances change
TC 8	View account statement	Select Print Statement	All transactions listed with timestamp & balance

6.2 Test Procedure

Each test case was executed manually by running the application from the console in IntelliJ IDEA, selecting the relevant menu option, entering the specified inputs, and comparing the displayed output and balance against the expected result. The application was also stopped and restarted between some tests to confirm that file-based persistence reloaded data correctly on the next run.

6.3 Performance Outcome

All 8 planned test cases passed with 100% success rate:

- Balances updated correctly after every deposit, withdrawal and transfer
- Invalid operations (wrong password, insufficient funds, invalid account) rejected with descriptive error messages
- Account and transaction data correctly reloaded after restarting the application
- Unique account number generation (UCT + random digits) produced no duplicates across multiple registrations

7. My Learnings

This internship gave me practical, end-to-end experience of building a Java application from a written problem statement. Key learnings include:

- Object-Oriented Design: Modelling real-world entities (customers, accounts, transactions) as Java classes with proper encapsulation.
- Exception Handling: Implementing custom exceptions for banking-specific errors and handling them gracefully without crashing.
- Java Collections: Using ArrayList and HashMap to manage multiple accounts in memory efficiently.
- File-based Persistence: Implementing Java File I/O to save and reload account and transaction data between application runs.
- Input Validation: Validating every user input — amounts, account numbers, passwords — before processing.
- Project Planning: Breaking a broad problem statement into a checklist of features and implementing them incrementally.
- Version Control: Using GitHub for the first time as part of an internship submission, managing a professional repository.

Beyond the technical skills, this internship taught me how industry-style problem statements work and how to convert requirements into working software.

8. Future Work Scope

Given more time and resources, this prototype could be extended in several directions:

- **Graphical User Interface (GUI):** Replace the console with a JavaFX or Swing-based GUI for a better user experience.
- **Database Integration:** Replace flat-file persistence with MySQL/PostgreSQL using JDBC for scalability.
- **Password Security:** Implement password hashing (e.g. BCrypt) instead of plain-text storage.
- **Multi-user / Admin Roles:** Add an administrator role to view all accounts and generate bank-wide reports.
- **Interest Calculation:** Implement automatic interest calculation on savings accounts.
- **Email/SMS Notifications:** Send transaction alerts via email or SMS using a notification API.
- **Loan Management Module:** Add ability to apply for, approve, and track loan repayments.
- **Web Interface:** Convert the application into a Spring Boot web application with REST APIs.

These enhancements would transform the current prototype into a production-grade banking application.